Conversational Recommender System

A multi-turn LLM shop assistant for smartphones & headphones — validated NLU, hybrid semantic retrieval, learned ranking & full observability

Group 26

Applied Generative AI · SS 2026

Arthur Galois · Jakob Meltzer · Sofie Parkesaas · Muhammad Azeem Shahzad

Abstract

We build a conversational recommender that acts as a shop assistant for tech products. Through multi-turn natural-language dialogue it elicits user preferences, reasons over an explicit dialogue state, and recommends concrete products. NLU is split into a routing + slot-filling pipeline guarded by schema validation and an entity-resolution layer; vague language (“cheap”, “good camera”) is grounded in dataset percentiles; retrieval blends structured pandas filters with dense-embedding semantic search for use-case (“vibe”) queries; products are ranked by TOPSIS, upgradable to a learned ranker via a click-feedback flywheel. A LangGraph state machine orchestrates four nodes; every turn is traced (LangSmith) and logged for analytics, and recommendations show real product photos.

Initial Situation & Problem

Traditional recommenders work silently (ranked lists, “people also bought”). Users often start vague and refine as they talk. A conversational recommender (CRS) must instead elicit preferences in real time and adapt.

Core challenges / questions:

- Robust intent detection & preference elicitation from messy natural language.
- Reasoning over a dialogue state to pick the next best action — not blindly answering.
- Safely translating conversational critiques (“cheaper but better camera”) into DB queries.
- What information does the system actually need to reliably recommend a set of items?

Sample Use Cases

1 · Specific lookup

“A Samsung Android phone with 8GB RAM”

Pre-fills brand+OS+spec, recommends immediately — ranked, no needless questions.

2 · Guided exploration

“I want a smartphone”

Asks a focused sequence: OS → price → battery → camera → RAM, with ‘any/skip’.

3 · Refinement / critique

“cheaper ones” · “bigger battery”

Anchors relative terms to the last results; refines are additive; ‘forget X’ removes.

4 · Vague & vibe language

“cheap phone for gaming”

Maps ‘cheap’→p25 price; ‘gaming’ via semantic embeddings over spec descriptions.

Approach — System Architecture

LangGraph 4-node pipeline · de-risked NLU · hybrid retrieval · learned ranking · observability

MEMORY
per-user profiles · ‘use my usual’ · personalised greeting

RANKING
TOPSIS (MCDM) ↔ learned LTR · semantic blend · feedback flywheel

OBSERVABILITY
LangSmith tracing · per-turn JSONL · intent & funnel analytics

IMAGES
offline enrichment → GSMArena/Wikipedia → image_url → cards

Key Design Decisions

- Split NLU into a small router + a slot extractor instead of one fragile mega-prompt.
- Validate every LLM-proposed slot against a schema; drop unknown / out-of-range / wrong-type.
- Entity resolution as a real layer (alias dictionary + fuzzy match), not ever-growing prompt rules.
- Explicit DialogueState (TypedDict) + LangGraph; information-gathering gate decides ask vs. recommend.
- Confidence-gated clarification; out-of-scope intent answers honestly instead of misfiring.
- Hybrid retrieval: exact pandas filters + dense embeddings (model2vec) for ‘vibe’ queries.
- Transparent TOPSIS ranking; pluggable learned ranker trained from click feedback.

Models & Tools

Orchestration	LangGraph state machine (4 nodes) + LangChain
LLM	OpenRouter · GPT-4o-mini (router, extractor, responder)
Embeddings	model2vec · potion-base-8M (static, no GPU)
Data / retrieval	pandas + numpy · 2×500 product catalog
Ranking	TOPSIS (Hwang & Yoon) · numpy logistic LTR
UI	Streamlit chat · comparison tables · product cards
Observability	LangSmith · structured JSONL logs
Images	GSMArena / Wikipedia / Openverse enrichment

Conversational Capabilities — intents → system actions

explore
category only → guided questions (OS→price→battery→camera→RAM)

specific
concrete criteria → recommend or one narrowing question

refine
react to results: ‘cheaper’ anchored to last set; additive merges

summarize
recap understood preferences in plain language

done
wrap-up / pick one / restart → clean close + new-search

chitchat
greetings/thanks → warm reply, product state preserved

ambiguous
too vague → ask to clarify (confidence-gated)

out_of_scope
photos/buy/stock/warranty → honest ‘can’t, but I can…’

Cross-cutting: skip (‘any’) · undo (‘ignore that’) · multi-intent · category-switch · vague-term grounding (percentiles) · persistent memory · quick-reply buttons.

Results, Demo & Learnings

95%	100%	100%	90%	100%	3.26s
Intent routing accuracy	Entity resolution accuracy	Slot validation (bad slots caught)	End-to-end task success	Phone image coverage	Avg latency per turn

Metrics from evaluate.py: 20 resolution + 10 validation (deterministic), 22 intent + 10 multi-turn scenarios (LLM-graded).

Demo — Example Conversation

You
a cheap android phone with a good camera

Assistant
Found 60 matches — top pick: Realme 9 Pro 5G. (price≤$147, camera≥64MP applied)

You
show me cheaper ones

Assistant
Narrowed to phones under $147, cheapest first. Want to compare a few?

You
can you show pictures of this?

Assistant
I can't show photos here, but I can share full specs or compare options.

Learnings & Limitations

What worked

- Schema validation eliminated a whole class of bugs (hallucinated / misplaced filters).
- Entity resolution was the single biggest robustness win (‘iphone’→apple, typos).
- Dense embeddings beat TF-IDF for ‘vibe’ queries (‘long flights’ ≈ noise-cancelling).
- Splitting NLU + an info-gathering gate made multi-turn behaviour reliable.

What was hard / we'd change

- LLMs occasionally emit a ‘null’ string or leak a percentile price cap — fixed with code guards.
- Three LLM calls/turn → ~10s latency; route with a smaller/faster model next.
- Free image sources miss budget brands — solved via GSMArena (100% phones; ToS caveat).
- Learned ranker needs real click data to beat TOPSIS; flywheel is built, awaiting traffic.

Evaluation Methodology

- Deterministic suites (free, instant): 20 entity-resolution cases (aliases, typos, synonyms) and 10 slot-validation cases (control fields, impossible values, unknown keys).
- LLM suites: 22 single-utterance intent cases (incl. prior-context turns) and 10 multi-turn scenarios graded by a state predicate (correct action + filters + non-empty results).
- Reproducible via `python evaluate.py` → eval_results.json; this poster renders those numbers.
- Iterative: the harness caught two real bugs (undo over-revert, fresh-search contamination) which were fixed — task success rose 70% → 90%.

Reflection — What does the system need?

- The product category — gates the schema, question order, and ranking weights.
- Structured preferences — extracted directly (specific) or elicited via questions (explore), then validated against the schema.
- An anchor for relative critiques — the previous recommendation’s distribution, to ground ‘cheaper’ / ‘bigger battery’.
- A multi-criteria scoring function — TOPSIS over normalised specs, upgradable to a learned ranker.
- Dataset statistics — percentiles that turn vague language (‘cheap’, ‘premium’) into concrete bounds.